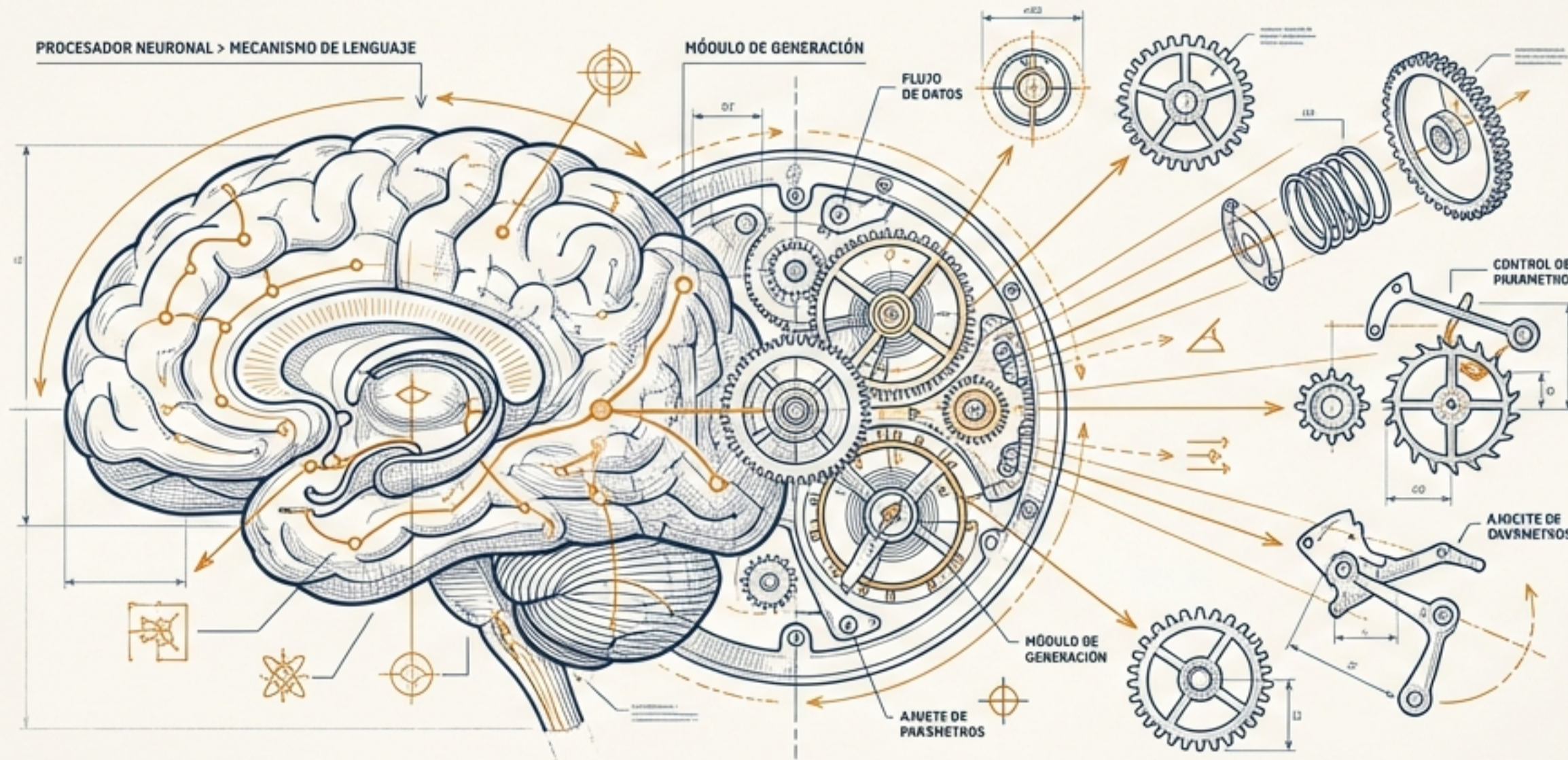


Deconstruyendo la Magia: Cómo Hablar con una IA Local



Bienvenida al mundo de la IA generativa en tu propia máquina.



Nuestro objetivo: programar una conversación con un modelo de lenguaje.



Aprenderemos dos formas de hacerlo: una manual y otra automatizada.



Entender la base teórica te convertirá en un mejor desarrollador.

El Escenario: Cliente, Servidor y el Puente llamado API



- **Servidor:** Ollama, ejecutándose en `localhost:11434`, esperando peticiones.
- **Cliente:** Nuestro script de Python (`prueba_a.py` o `prueba_b.py`), que necesita una respuesta del modelo.
- **API:** El conjunto de reglas y formatos que permite que el cliente y el servidor se entiendan.
- Nuestro script no 'controla' al modelo, le 'pide' cosas a través de la API.

Camino 1: La Vía Directa con HTTP y `requests`

- Hablaremos con la API directamente, sin intermediarios.
- Usaremos la librería `requests` de Python, el estándar para hacer peticiones HTTP.
- Construiremos la petición manualmente, definiendo cada parte.
- Esto nos da control total, pero requiere conocer los detalles del protocolo.

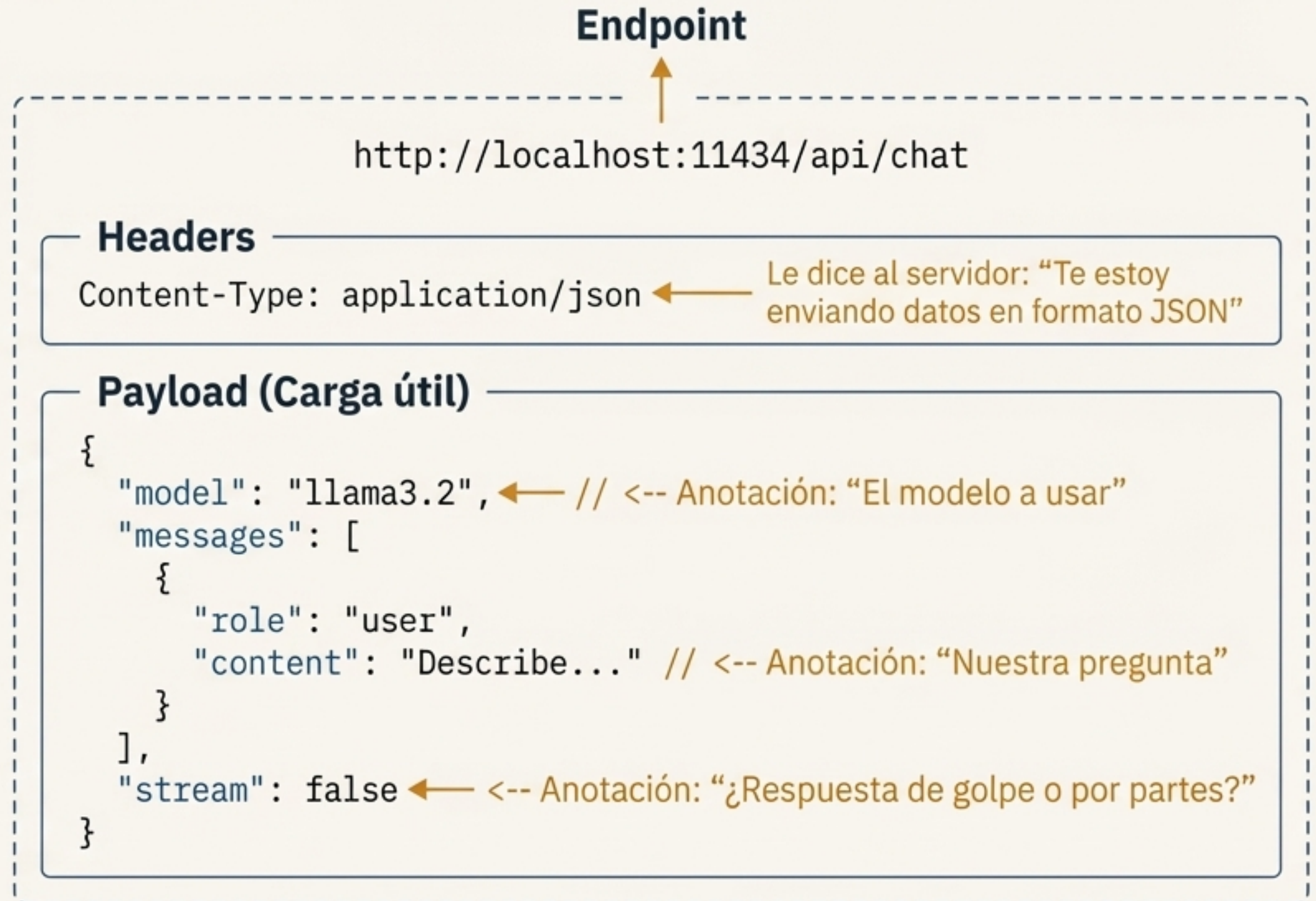


Analogía: Es como escribir una carta formal. Necesitas un sobre (petición HTTP), una dirección (URL), y el contenido en un formato específico (JSON). La librería `requests` es el servicio de correos que la entrega.



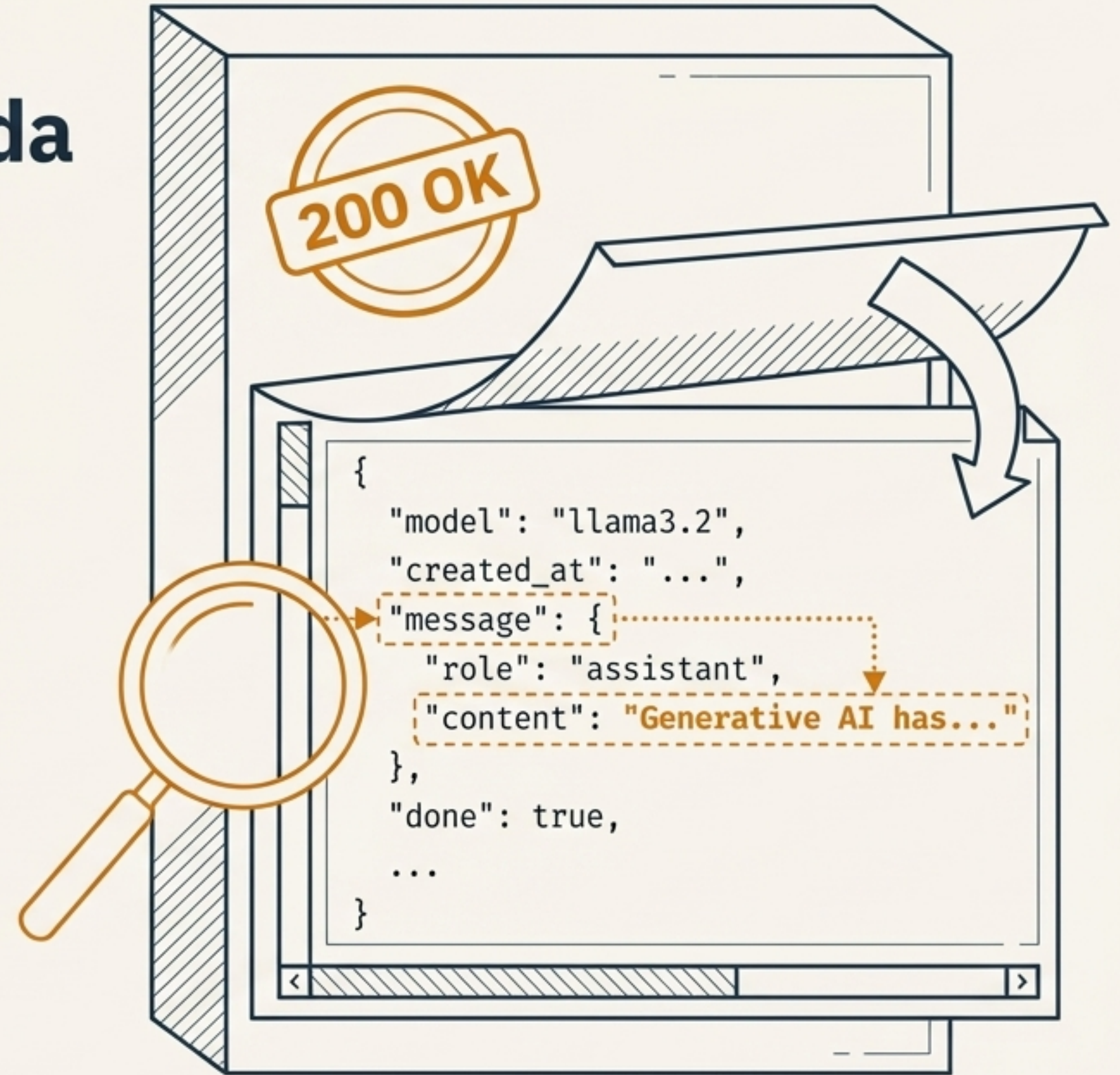
Anatomía de Nuestra Petición: JSON, el Idioma Universal

- **Endpoint:** La URL específica a la que enviamos la petición.
- **Headers:** Metadatos para la comunicación.
- **Payload:** Los datos en sí, estructurados en formato JSON.
- **JSON:** El formato de texto para intercambiar datos, basado en pares clave-valor.



La Respuesta del Servidor: Navegando el JSON de Salida

- El servidor responde con un código de estado HTTP (esperamos `200 OK`).
- La respuesta exitosa contiene un cuerpo (body) en formato JSON.
- El texto generado por el modelo está "anidado" dentro de esta estructura.
- Accedemos a las claves para extraer el contenido:
``response['message']['content']``



Camino 2: La Vía Rápida con el SDK de Ollama

- Un SDK (Software Development Kit) es un conjunto de herramientas que simplifica la interacción con una API.
- En lugar de construir peticiones HTTP, llamamos a funciones de Python como ``ollama.chat()``.
- El SDK se encarga de la comunicación 'sucias' por debajo.
- El código es más limpio, más corto y más fácil de leer.



Analogía: En lugar de escribir la nota del pedido a mano (API), usas una app en una tablet (SDK). Seleccionas las opciones y pulsas "Enviar". La app se encarga del resto.



SDK vs. API Directa: ¿Cuándo usar cada uno?

API Directa (requests)



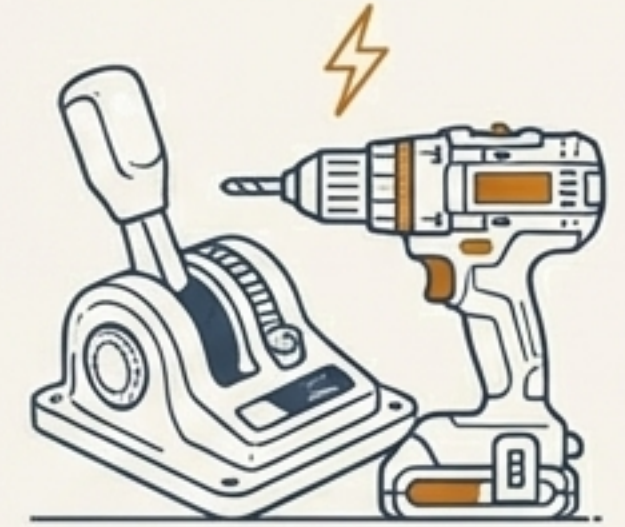
Ventajas:

- Control total sobre cada detalle.
- Sin dependencias externas (solo `requests`).
- Ideal para aprender y depurar.

Desventajas:

- Más código repetitivo (`boilerplate`).
- Más propenso a errores manuales.
- Tú gestionas la complejidad.

SDK (ollama)



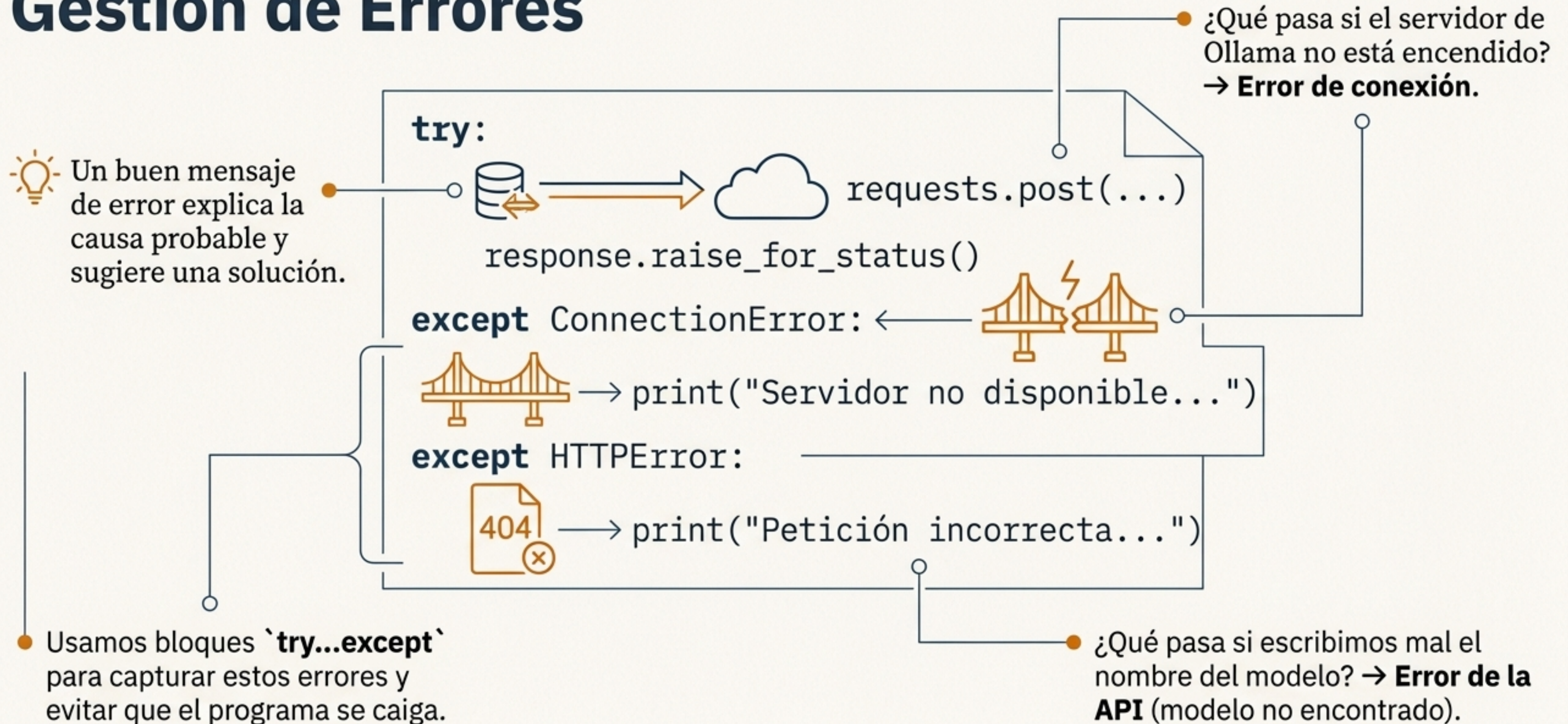
Ventajas:

- Código simple y rápido de escribir.
- Menos propenso a errores comunes.
- Gestiona la complejidad por ti.

Desventajas:

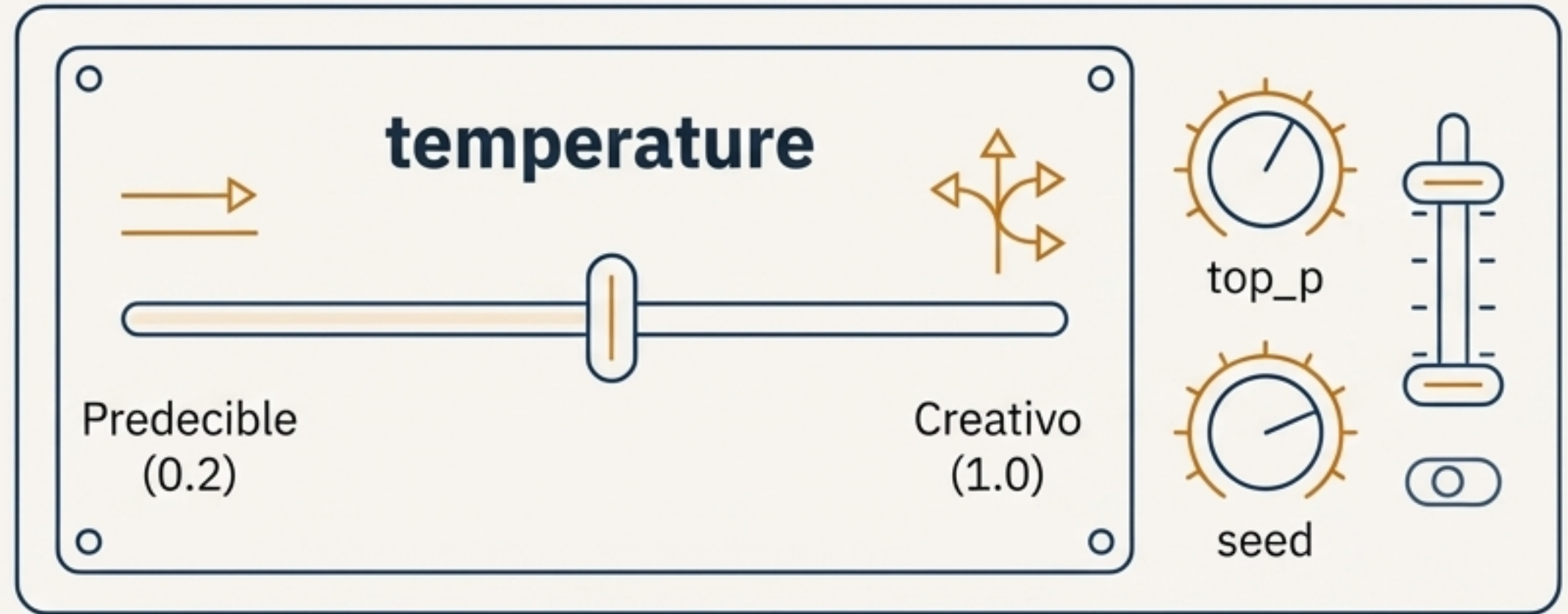
- Añade una dependencia al proyecto.
- Menos control sobre detalles finos.

Preparándonos para lo Inesperado: Gestión de Errores



Personalizando la IA: El Poder de las `options`

- La IA no es una caja negra; podemos influir en su comportamiento.
- El parámetro `options` nos permite ajustar la generación.
- `temperature`: Controla la “creatividad” vs. “determinismo”.
- Otros parámetros pueden controlar el estilo, longitud y formato.



```
"payload": {  
  "model": "llama3.2",  
  "messages": [...],  
  "stream": false,  
  "options": {  
    "temperature": 0.9 // <-- Highlighted in ochre-orange  
  }  
}
```


Conclusiones Clave



- Hemos aprendido a comunicarnos con una IA local a través de su API.
- Entendimos la vía manual (HTTP/JSON) que nos da control y conocimiento.
- Descubrimos la vía rápida (SDK) que nos da productividad y simplicidad.
- Un buen desarrollador conoce ambas y elige la herramienta adecuada para cada trabajo.
- La gestión de errores y la personalización son claves para crear aplicaciones robustas y eficaces.

Posibles Errores Comunes (¡A evitar!)



Ollama no está en ejecución: El error más común (`ConnectionError`). Verificar que el servicio esté activo.



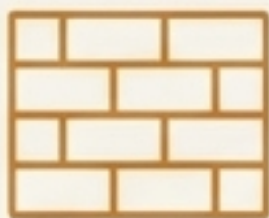
Nombre del modelo incorrecto o no descargado: Asegurarse de haber hecho `ollama pull llama3.2`.



URL del endpoint incorrecta: Un `/` de más o de menos puede causar un error 404 Not Found.

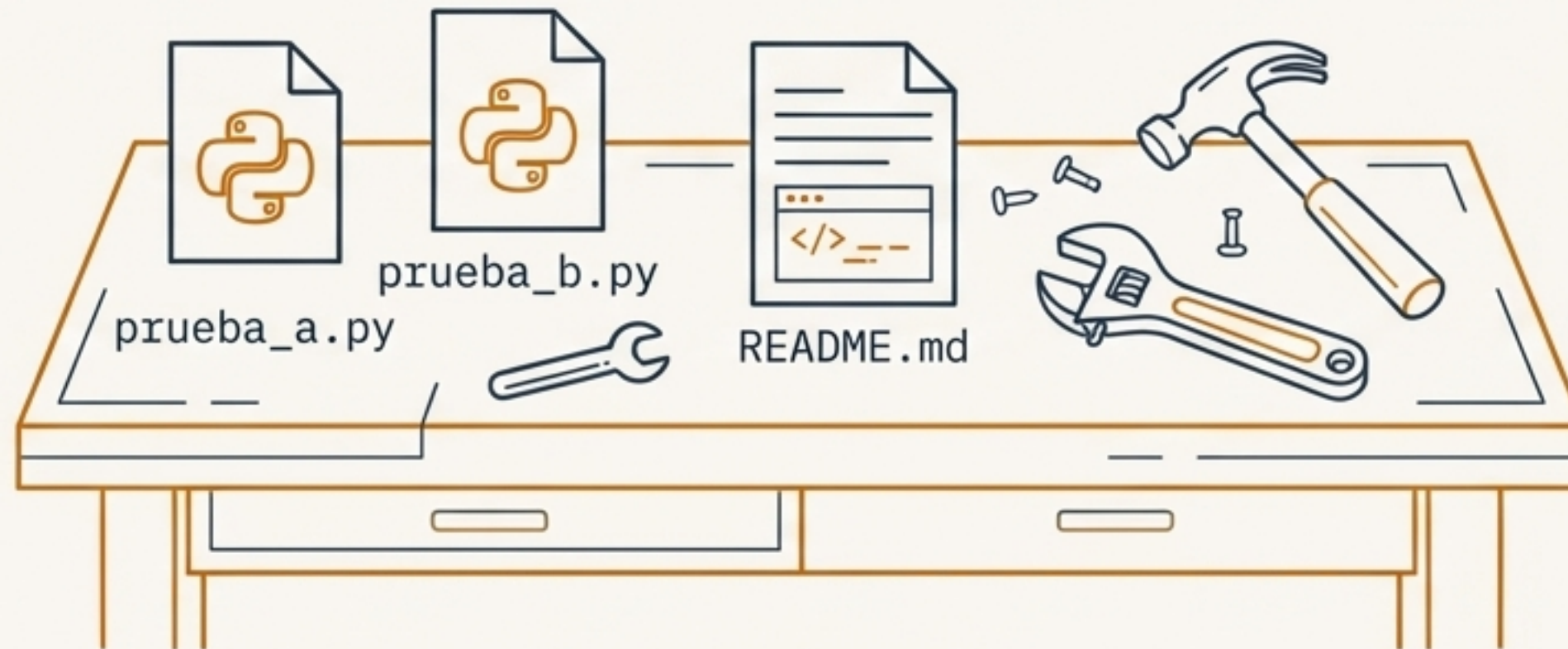


Error de sintaxis en el JSON: Una coma mal puesta o una comilla faltante invalidará el payload.



Firewall bloqueando `localhost`: Menos común, pero posible en entornos corporativos.

¡Ahora te Toca a Ti! Próximos Pasos



- Completa las prácticas **prueba_a.py** y **prueba_b.py**.
- Responde a las preguntas de reflexión en tu README.
- ¡No copies y pegues! El verdadero aprendizaje está en rellenar los huecos (**TODO**).



Challenge Section

Reto extra: Intenta implementar las **mejoras opcionales** (argumentos por línea de comandos, guardar en archivo, medir el tiempo).